



中期小结实验报告

双截龙横版动作游戏

C++ / Qt / MVVM 架构设计与实现

课程名称	C++ 项目管理及工程实践
小组名称	双截龙组
小组成员	苏易文 (App/Common 层) 陈棋隆 (View 层) 邱安琪 (ViewModel 层)
完成日期	2026 年 7 月 14 日

目录

1 实验概述	1
1.1 组内分工	1
1.2 协作与版本管理	1
1.3 团队协作情况	3
1.4 当前完成情况	4
1.5 阶段性成果展示	5
2 总体架构设计	6
2.1 架构目标	6
2.2 依赖方向	6
2.3 绑定方式	7
2.4 数据流设计	8
2.5 时间推进机制	8
2.6 状态边界	8
3 App/Common 层分报告	9
3.1 层次定位	9
3.2 Common 层设计	9
3.3 App 层设计	11
3.4 技术难点的克服	11
3.5 小结	13
4 View 层分报告	14
4.1 层次定位	14
4.2 文件组织与编译	14
4.3 MainWindow 主窗口	15
4.4 GameWidget 游戏画布	17
4.5 InputDefs.h 输入聚合	22
4.6 与旧版设计的对比	23
4.7 小结	24
4.8 技术难点的克服	24
5 ViewModel 层分报告	26
5.1 层次定位	26
5.2 文件组织与编译	26
5.3 GameViewModel	27

5.4 GameSimulation 仿真核心	28
5.5 状态快照与阶段切换	29
5.6 小结	30
5.7 技术难点的克服	30
6 个人反思与收获	31
6.1 苏易文 - App/Common 层	31
6.2 陈棋隆 - View 层	32
6.3 邱安琪 - ViewModel 层	32
7 总结与展望	33
7.1 架构收获	33
7.2 项目整体进展的经验体会	33
7.3 智能体的使用	34
7.6 后续改进方向	35
7.7 总结	35

1 实验概述

本实验仓库实现的是一款横版街机动作游戏，主题为“双截龙”。项目使用 C++17、Qt6、CMake 与 vcpkg 组织工程，当前代码已经形成了较清晰的 MVVM 分层：common 层提供跨层公共状态和通知工具，viewmodel 层负责游戏规则和状态推进，view 层负责 Qt 窗口、输入和绘制，app 层作为组合根管理生命周期以及属性、命令、通知三类绑定。

从玩法目标看，玩家扮演一名格斗角色，在带有纵深的城市街道上从左向右推进。当前实现支持标题画面、移动、跳跃、轻攻击、重攻击、血量、精力、疲劳提示、普通敌人追击、Boss 触发、暂停、失败、胜利和重开等核心流程。普通敌人在初始阶段生成并主动靠近玩家；当玩家推进到关卡后段时，系统生成 Boss，击败 Boss 后进入胜利结算。

1.1 组内分工

本组以层次边界作为分工单位，每位成员既负责对应代码，也负责对应分报告：

- 苏易文：负责 App/Common 层以及最终报告的撰写，分报告为 02_app_common_layer.md，主要内容包括生命周期、组合根、公共状态、通知工具和三类绑定。
- 陈棋隆：负责 View 层，分报告为 03_view_layer.md，主要内容包括窗口、画布、输入、HUD 和覆盖层。
- 邱安琪：负责 ViewModel 层，分报告为 04_viewmodel_layer.md，主要内容包括命令回调、状态推进、AI、战斗和胜负判定。

1.2 协作与版本管理

为了体现组内协作过程，本项目在开发中持续使用 Git 和 GitHub 记录提交历史。三位成员围绕不同层次并行推进：Common/App 层先确定公共状态和绑定方式，View 层根据公共状态完成输入和绘制，ViewModel 层独立实现规则模拟和状态同步，最后通过主分支合并和冲突处理完成整合。

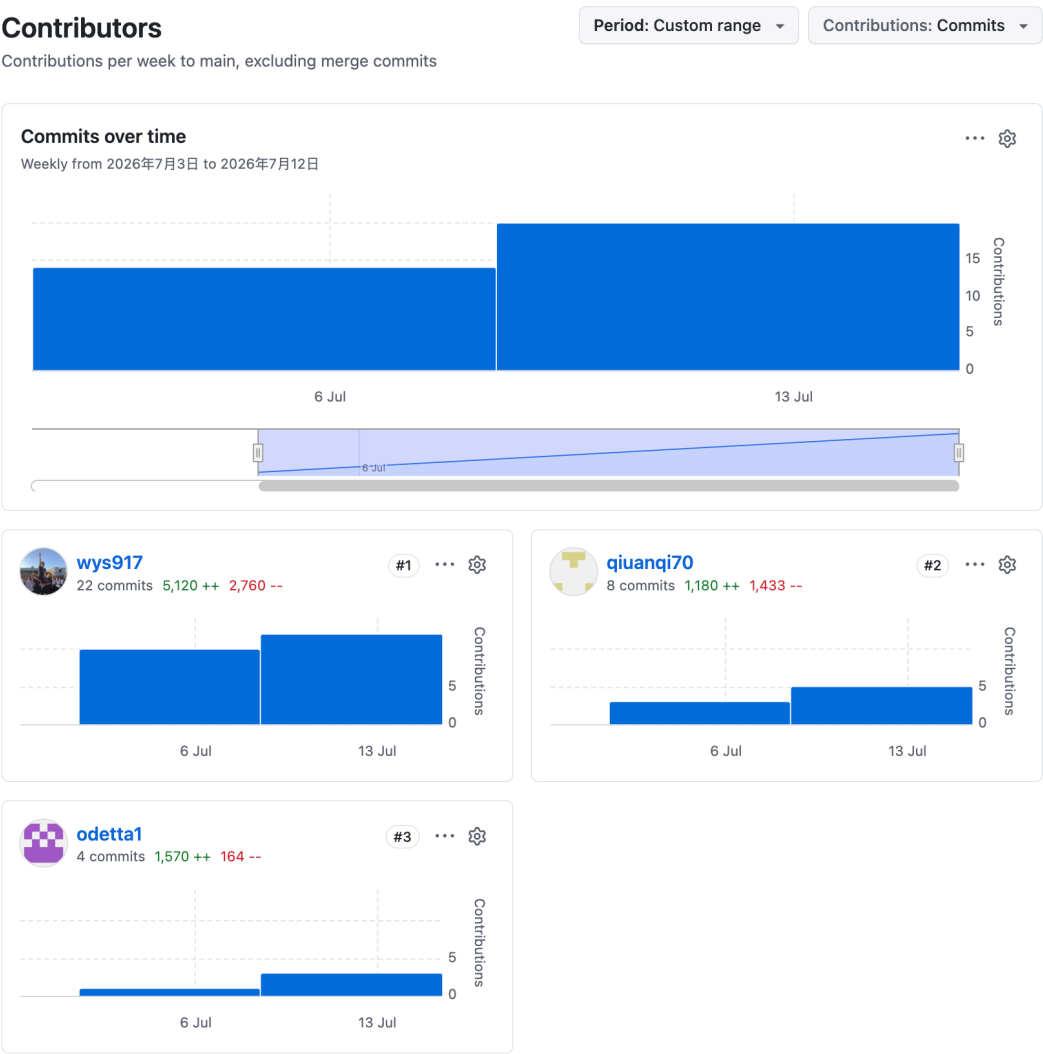


图 1 GitHub Contributors 统计图

图中的 GitHub 账号与组员对应关系如下：wys917 对应苏易文，qiuqianqi70 对应邱安琪，odetta1 对应陈棋隆。从 GitHub Contributors 页面可以看到三位成员均有提交记录，说明项目并不是由单人集中完成，而是按照分层职责持续推进。

需要补充说明的是，上图中统计的提交数和增删行数不能等同于实际工作量，本次任务中我们小组全程保持顺畅的沟通，每个人都很负责的完成了自己的部分，并在有余力时对队友的工作提出建议或帮助，三人的工作量是一致的。



图 2 Git 提交历史与协作解耦记录

上图合并展示了项目的提交历史。可以看到仓库中存在多次围绕 viewmodel、view、common 和 app 的独立提交与分支合并，并不是在同一个大文件中串行修改，而是按照 MVVM 层次拆分任务，并持续进行解耦：把公共契约从具体规则中分离出来，让 View 和 ViewModel 可以围绕稳定的 GameState、命令回调和通知机制并行开发，最后再通过 merge 将各层成果整合到主分支。

1.3 团队协作情况

本组整体沟通与合作较为顺畅。三位成员按照 MVVM 层次划分职责，在明确各自代码边界的基础上并行开发，并通过线下讨论、文档说明和 Git 提交及时同步接口变化与实现进度。Common/App 层负责确定共享数据结构和绑定方式，View 层围绕稳定的 GameState 完成输入与绘制，ViewModel 层

独立推进游戏规则和状态同步，最终由 App 层集中完成 `properties / commands / notification` 的组装与联调。这样的协作方式减少了多人同时修改同一文件的情况，也使各成员能够专注于自己负责的模块。

协作中的主要亮点体现在以下几个方面：

- **分工边界清晰。**各成员以 App/Common、View、ViewModel 为责任单元，代码文件和分报告均与分工对应，既便于独立推进，也便于出现问题时快速定位负责模块。
- **接口能够及时对齐。**Common 层通过 GameState、EventNotification 等公共契约统一跨层数据，ViewModel 暴露的命令接口与 View 的 setter 接口保持对应，降低了联调阶段因命名和职责理解不一致产生的沟通成本。
- **成员之间能够互相配合。**当公共类型、绑定方式或状态字段发生调整时，成员会及时沟通并共同完成适配；在完成各自任务后，也会对其他模块的设计和实现提出建议，而不是只关注个人代码。
- **版本管理较为规范。**三位成员均通过 Git 提交自己的阶段性成果，并通过分支合并完成整合。提交历史能够反映各层的开发与解耦过程，也为出现问题后的回溯和比较提供了依据。

目前的协作方式仍有进一步改进的空间。项目初期公共类型的命名和字段组织经历了多次调整，导致依赖这些接口的 View 和 ViewModel 需要重复适配；后续可以在编码前先形成简要的接口文档，并在修改公共契约前由全组确认。当前代码审查主要依靠线下讨论，缺少较系统的 Pull Request 审查和检查清单，部分潜在问题可能只能在最终联调时发现。此外，自动化测试主要集中在 ViewModel，跨层绑定、键盘输入和界面绘制仍以人工验证为主。后续若增加固定的阶段同步、交叉代码审查和基础集成测试，可以进一步降低返工成本，提高团队协作的稳定性与工程质量。

1.4 当前完成情况

当前仓库已经完成了游戏的基本可运行骨架和核心流程：

- **工程层面：**通过 CMake 分 target 组织 `alleyfist_common`、`alleyfist_viewmodel`、`alleyfist_view`、`alleyfist_app` 和最终可执行文件 `AlleyFist`。
- **公共契约：**使用 `types.h` 定义 `Size`、`WorldPosition`、`ResourceBar` 等基础值类型；使用 `game_state.h` 定义 `GameState`、`ActorState`、`MapState`、`HudState` 等只读显示状态；使用 `notification.h` 提供 `EventTrigger` 与 `EventNotification`。
- **ViewModel：**通过 `GameViewModel` 对外暴露属性和命令，通过 `GameSimulation` 维护实体列表和规则推进，支持玩家移动、跳跃、攻击、精力恢复、普通敌人追击、Boss 生成、失败和胜利判定。
- **View：**实现 60 FPS 定时器、键盘输入映射、世界坐标到屏幕坐标的投影、街道背景、视差建筑、角色几何绘制、血条、精力条、Boss 血条、进度条、标题画面和胜负覆盖层。
- **App：**通过 `GameApplication` 创建 `QApplication`、`GameViewModel` 和 `MainWindow`，并集中完成 `properties / commands / notification` 绑定。

1.5 阶段性成果展示

当前版本已经完成从标题画面、关卡战斗到胜负结算的主要交互流程。为了直观展示本阶段的开发成果，下面按照实际游戏流程选取六张运行截图，并采用每行三张的方式集中呈现。



图 3 AlleyFist 当前阶段主要运行界面

各截图反映的阶段性成果如下：

- **开始界面 (a)：**游戏启动后首先进入标题阶段，画面展示项目名称、中文主题、主要操作键位和开始提示。玩家按 Enter 后即可进入正式游戏，说明标题状态与确认命令已经完成联动。
- **普通战斗场景 (b)：**进入游戏后，玩家可以在带纵深的街道中移动并与多个普通敌人战斗。界面顶部显示玩家生命值和精力，底部显示关卡推进进度，体现了角色绘制、敌人 AI、战斗规则、HUD 与镜头跟随等功能的综合效果。
- **Boss 战场景 (c)：**当玩家推进到关卡后段时，系统会在前方生成体型和血量更高的 Boss，并在界面顶部显示独立的 Boss 血条。该画面说明关卡触发条件、Boss 实体生成、特殊角色绘制和首领状态同步已经打通。
- **暂停界面 (d)：**玩家按 P 或 Esc 后，游戏进入暂停阶段并显示半透明覆盖层，同时保留暂停前的场景和 HUD。再次按键即可继续游戏，验证了阶段切换、时间推进控制和覆盖层绘制功能。
- **失败结算界面 (e)：**玩家生命值清空后进入 Game Over 状态，界面显示存活时间和击败敌人数量，并允许玩家重新开始。这表明生命判定、统计数据、失败状态切换和重开流程已经形成完整闭环。
- **胜利结算界面 (f)：**玩家击败 Boss 后进入胜利状态，画面展示通关提示、完成时间和击败数量。该界面直观体现了 Boss 击败判定、胜利结算以及整局游戏流程的最终成果。

以上截图表明，项目已经形成可连续体验的游戏原型，并打通了输入响应、规则推进、界面更新和流程切换，为后续完善动画、音效、连招及关卡内容奠定了基础。

2 总体架构设计

2.1 架构目标

本项目选择 MVVM 风格的分层设计，主要目标有三个：

- 1. View 只处理“如何显示”和“如何接收输入”，不保存玩法规则状态。
- 2. ViewModel 只处理“游戏如何推进”和“当前状态是什么”，不依赖 Qt 控件或绘图 API。
- 3. Common 层只定义两边共享的数据形状和轻量通知工具，让 View 与 ViewModel 可以通过稳定合同通信。

这个设计使得界面、规则和应用启动逻辑分别处在不同 target 中。后续无论是替换绘制方式、扩展敌人 AI，还是调整应用启动流程，都不需要牵动整个项目。

2.2 依赖方向

项目的依赖方向如下：

```
main
-> app
    -> view
    -> viewmodel
    -> common
    -> Qt6 Core / Widgets
view
-> common
-> Qt6 Widgets / Gui
viewmodel
-> common
common
-> C++ standard library only
```

app 层可以看到具体的 View 和 ViewModel，因为它承担组合根职责；但 view 和 viewmodel 之间没有直接依赖。两者通过 Common 中的 GameState、EventNotification 以及 App 注入的 std::function 命令回调完成协作。

层次	主要文件	对外暴露	不应承担的职责
Common	types.h、 game_state.h、 notification.h、 common.h	基础值类型、只读游戏状态、事件通知工具	Qt 绘制、AI、碰撞、生命周期管理

层次	主要文件	对外暴露	不应承担的职责
ViewModel	GameViewModel、 GameSimulation、 SimulationTypes.h	const GameState*、 命令回调、变化通知	直接绘制画面、读取键盘码
View	MainWindow、 GameWidget、 InputDefs.h	属性 setter、命令 setter、 Qt 窗口与画布	修改游戏内部状态、计算伤害
App	GameApplication	run()	游戏规则、具体绘制细节

2.3 绑定方式

当前项目参考袁老师的 Plane 示例，将跨层绑定拆成三类。

第一类是属性绑定：

```
GameViewModel::get_game_state()
-> const GameState*
-> MainWindow::set_game_state(...)
-> GameWidget::set_game_state(...)
-> paintEvent 读取只读状态
```

View 持有的是 const GameState*，只能读，不能写。GameState 中包含阶段、地图、玩家、敌人、HUD、结算等绘制所需信息。

第二类是命令绑定：

```
GameViewModel::get_move_left_command()
-> std::function<void(bool)>
-> MainWindow::set_move_left_command(...)
-> GameWidget::keyPressEvent / keyReleaseEvent 调用
```

移动命令使用 std::function<void(bool)> 表达按下和释放；轻攻击、重攻击、跳跃、暂停、确认、重置等单次动作使用 std::function<void()>；Tick 使用 std::function<void(float, std::uint64_t)>。这样 ViewModel 不需要知道用户按的是方向键、WASD、J 还是 Z，View 也不需要知道命令内部如何修改规则状态。

第三类是通知绑定：

```
GameViewModel::fire(kGameStateChangedEvent)
-> EventNotification
-> MainWindow 返回的 lambda
-> GameWidget::update()
```

状态变化后，ViewModel 只发出“状态已变”的通知。View 收到通知后安排重绘，并在 paintEvent 中通过之前绑定的只读指针读取最新 GameState。

2.4 数据流设计

项目中存在两条单向数据流。

第一条是输入和时间通道：

```
Qt key event
-> GameWidget
-> std::function command
-> GameViewModel
-> GameSimulation
```

View 将物理输入翻译成语义命令，例如“左移动开启”“轻攻击触发”“暂停触发”。

第二条是状态通道：

```
GameSimulation
-> GameViewModel::sync_state_from_simulation()
-> GameState
-> EventTrigger 通知
-> GameWidget::paintEvent
```

ViewModel 将内部实体状态整理为 **GameState**。**View** 只根据其中的 `phase`、`map`、`player`、`enemies`、`hud`、`result`、`screenMessage` 等字段绘制画面。

2.5 时间推进机制

游戏循环由 **View** 层的 `QTimer` 驱动，约每 16 ms 触发一次。触发时，`GameWidget` 根据 `QElapsedTimer` 计算真实 **delta time**，并将单帧推进限制在 0.1 秒以内，避免调试或卡顿后一次性推进过多逻辑。随后 `GameWidget` 调用已绑定的 `m_tickCommand(clampedDt, frameIndex)`。

2.6 状态边界

GameState 是 **ViewModel** 到 **View** 的主要状态出口。当前状态中包含：

- **GamePhase**：标题、游玩、暂停、失败、胜利等阶段。
- **MapState**：视口大小、镜头位置、街道上下边界。
- **ActorState**：玩家、敌人、Boss 的位置、绘制尺寸、血量、动作状态、朝向和可见性。
- **HudState**：玩家血量、玩家精力、Boss 血条和疲劳提示。
- **GameResultState**：用时和击败敌人数。
- **screenMessage**：标题或暂停等阶段的屏幕提示。

内部的实体列表、敌人追踪规则、Boss 触发位置、玩家移动标志、跳跃计时、攻击计时和能量消耗常量都保留在 **ViewModel** 内部，不向 **View** 暴露。这使得 **View** 可以保持稳定，即使后续增加更复杂的 AI、连招或关卡配置，也只需要在 **GameState** 中补充必要显示字段。

3 App/Common 层分报告

负责人：苏易文 学号：3240103466

3.1 层次定位

App/Common 这一部分承担两个不同但相关的职责。

common/ 是 View 与 ViewModel 之间的公共合同。它不依赖 Qt，也不写具体玩法，只定义两层都能理解的基础值类型、只读游戏状态和通知工具。app/ 是应用组合根，负责创建 Qt 应用对象、主窗口和 ViewModel，并把 View 与 ViewModel 通过属性、命令和通知三类接口绑定起来。

这两层的共同目标是控制依赖边界：公共状态和通知工具放在 Common，具体对象创建和绑定放在 App，避免 View 和 ViewModel 彼此直接包含对方头文件。

3.2 Common 层设计

Common 层主要由四类文件组成：

文件	作用
types.h	定义 Size、WorldPosition、ResourceBar 等基础值类型
game_state.h	定义 GameState 及其子结构，作为 View 可读取的状态协议
notification.h	定义 EventNotification 与 EventTrigger，作为变化通知工具
common.h	便捷入口，统一包含 Common 层公共头文件

3.2.1 基础值类型

types.h 中只保留跨层都会用到、但不携带具体玩法规则的基础属性：

```
struct Size {
    float width = 0.0f;
    float height = 0.0f;
};

struct ResourceBar {
    int current = 0;
    int maximum = 0;

    float ratio() const noexcept;
    bool is_empty() const noexcept;
};
```

```
struct WorldPosition {  
    float x = 0.0f;  
    float laneY = 0.0f;  
    float z = 0.0f;  
};
```

这些类型可以被 **View** 和 **ViewModel** 同时理解。**WorldPosition** 中 *x* 表示横向推进, *laneY* 表示街道纵深, *z* 表示离地高度; **View** 用它投影到屏幕坐标, **ViewModel** 用它记录实体位置。

3.2.2 游戏状态

`game_state.h` 将 **View** 需要读取的状态拆成几个层次:

- **GamePhase** 描述当前阶段, 包括 **Title**、**Playing**、**Paused**、**GameOver** 和 **Win**。
- **MapState** 描述视口、镜头和街道边界。
- **ActorState** 描述单个角色, 包括阵营、种类、位置、绘制尺寸、血量、动作状态、朝向和可见性。
- **HudState** 整理 HUD 专用数据, 包括玩家血量、精力、Boss 血条和疲劳状态。
- **GameResultState** 记录结算数据, 包括用时和击败数量。
- **GameState** 汇总完整帧状态, 作为 **View** 绘制一帧画面的主要输入。

这样的设计避免 **View** 从多个内部对象中拼接信息。**View** 只需要读取 **GameState**, **ViewModel** 负责把内部规则状态整理成适合显示的数据。

3.2.3 事件通知

`notification.h` 提供一个与 Qt 无关的轻量通知工具:

```
using EventNotification = std::function<void(std::uint32_t eventId)>;  
  
class EventTrigger {  
public:  
    std::uintptr_t add_notification(EventNotification&& notification);  
    void remove_notification(std::uintptr_t cookie) noexcept;  
  
protected:  
    void fire(std::uint32_t eventId);  
};
```

ViewModel 继承 **EventTrigger**, 状态变化后调用 `fire(kGameStateChangedEvent)`; **View** 通过 `MainWindow::get_notification()` 提供一个闭包, 收到通知后调用 `GameWidget::update()`。通知只表达“某类事件发生了”, 不携带 Qt 对象, 也不暴露 **ViewModel** 内部对象。

3.3 App 层设计

GameApplication 是 App 层对外暴露的生命周期接口。它使用 PImpl 隐藏 Qt、主窗口和 ViewModel 的具体类型，公共头文件中只保留构造、析构和 run()。

实现层中，GameApplication::Impl 创建三个核心对象：

```
QApplication qtApp;  
GameViewModel viewModel;  
MainWindow mainWindow;
```

随后在构造函数中集中完成三类绑定。

第一类是属性绑定：

```
mainWindow.set_game_state(viewModel.get_game_state());
```

第二类是命令绑定：

```
mainWindow.set_tick_command(viewModel.get_tick_command());  
mainWindow.set_move_left_command(viewModel.get_move_left_command());  
mainWindow.set_move_right_command(viewModel.get_move_right_command());  
mainWindow.set_move_up_command(viewModel.get_move_up_command());  
mainWindow.set_move_down_command(viewModel.get_move_down_command());  
mainWindow.set_primary_action_command(viewModel.|  
    ↪ get_primary_action_command());  
mainWindow.set_secondary_action_command(viewModel.|  
    ↪ get_secondary_action_command());  
mainWindow.set_state_toggle_command(viewModel.get_state_toggle_command|  
    ↪ ());  
mainWindow.set_reset_command(viewModel.get_reset_command());  
mainWindow.set_confirm_command(viewModel.get_confirm_command());  
mainWindow.set_pause_command(viewModel.get_pause_command());
```

第三类是通知绑定：

```
viewModel.add_notification(mainWindow.get_notification());
```

App 层知道 MainWindow 和 GameViewModel 的具体类型，但绑定结果仍然保持 View 与 ViewModel 的单向依赖：View 只持有只读状态指针和命令闭包，不包含 viewModel/ 头文件；ViewModel 只保存通知闭包，不知道具体的 GameWidget。

3.4 技术难点的克服

App/Common 层位于多个模块的交界处，其技术难点不仅在于完成自身代码，还在于建立一套能被 View 和 ViewModel 共同接受并长期使用的公共边界。开发过程中遇到的主要挑战及协作解决过程如下。

3.4.1 公共契约的职责收敛

项目初期, Common 层曾包含较多带有业务含义的类型, 状态结构的命名和字段组织也经历了多次调整。例如, 部分类型先后使用过 ViewData、Snapshot 和 State 等命名, 输入动作、状态快照和变化比较逻辑也一度被放入公共层。这样的设计虽然能快速连接各模块, 但会使 Common 同时承担 View 输入、ViewModel 规则和数据同步等职责。一旦公共结构发生变化, View 的绘制代码和 ViewModel 的状态转换代码都需要同步修改, 放大了接口变更的影响范围。

为解决这一问题, App/Common、View 和 ViewModel 三位成员共同梳理了各层真正需要共享的信息。View 层列出绘制一帧所需的位置、尺寸、血量、动作状态和 HUD 数据, ViewModel 层确认哪些模拟参数应继续保留在内部, App/Common 层再据此精简公共接口。经过多轮讨论与适配, Common 最终收敛为 types.h、game_state.h 和 notification.h 三类稳定内容: 基础值类型、只读显示状态和轻量事件通知; 移动速度、攻击范围、Boss 触发条件等规则数据则保留在 ViewModel 内部。团队还通过 README、代码注释和 Git 提交同步接口变化, 使各模块能够在统一契约下继续开发。虽然前期调整带来了一定返工, 但最终显著降低了跨层耦合, 也让后续新增状态字段时能够更快判断其所属层次。

3.4.2 绑定机制的解耦与统一

第二个难点是如何在不让 View 和 ViewModel 直接依赖彼此的前提下完成交互。早期方案倾向于使用 Qt 的 signal/slot 和额外绑定器连接两层, 这种方式实现方便, 但会让绑定过程依赖 Qt 类型, 并容易把界面事件、业务命令和对象生命周期混合在一起, 不利于保持 ViewModel 的纯 C++ 边界。同时, 属性读取、输入命令和状态通知采用不同机制, 也增加了联调和排查问题的难度。

团队参考课程示例后, 统一采用 properties / commands / notification 三类绑定。ViewModel 层负责提供只读 GameState 指针和 std::function 命令, View 层提供与命令对应的 setter 以及重绘通知闭包, App/Common 层则在 GameApplication::Impl 中集中完成全部注入。改造过程中, View 与 ViewModel 成员先共同核对移动、攻击、暂停、确认和重置等命令的名称与参数, 再由 App 层逐项连接并进行联合编译和运行验证。最终, View 不需要包含 ViewModel 头文件, ViewModel 也不需要依赖 Qt 控件; 若绑定出现遗漏, 只需检查组合根中的集中绑定代码, 问题定位比原先更加直接。

3.4.3 跨模块联调与版本冲突处理

App/Common 的改动通常会同时影响 View 和 ViewModel, 因此第三个难点是控制并行开发中的编译错误和合并冲突。尤其在公共类型重命名、状态字段重组和绑定接口调整期间, 不同分支可能仍引用旧接口, 单个模块能够编译并不代表合并后的完整程序可以正常运行。此外, 属性指针、命令闭包和通知回调只有在 App 层全部组装后才能验证完整的数据流, 问题往往集中暴露在最终联调阶段。

对此, 团队按照“先稳定公共契约, 再分别适配, 最后集中集成”的顺序推进: App/Common 层先提交接口调整并说明变化, View 和 ViewModel 成员分别通过全局搜索清理旧类型和旧函数, 随后在主分支完成合并与冲突处理。联调时, 三位成员沿着“键盘输入—命令回调—规则推进—GameState 同步—通知重绘”的数据流逐段检查, 而不是只在各自模块内部排查。Git 提交历史保留了公共边界精

简、耦合修复和合并冲突处理的过程，出现回归时可以快速比较修改前后的接口。通过这种分层排查和共同验证，团队最终打通了标题、游戏、暂停、失败、胜利和重开等完整流程，也积累了在多人协作中先约定接口、再并行实现、最后统一集成的实践经验。

3.5 小结

App/Common 层建立了稳定边界：Common 把基础值类型、只读状态和通知工具抽象成跨层协议；App 把具体对象创建和三类绑定集中在组合根中。这样整个项目的依赖方向保持清晰，也为 View 和 ViewModel 分别开发提供了基础。

4 View 层分报告

负责人：陈棋隆 学号：3240103461

4.1 层次定位

View 层负责” 玩家看到什么” 和” 玩家输入了什么”。它由主窗口 `MainWindow`、游戏画布 `GameWidget` 和输入聚合定义 `InputDefs.h` 组成, 主要使用 Qt 的窗口 (`QMainWindow`)、定时器 (`QTimer`)、键盘事件 (`QKeyEvent`) 和 `QPainter` 完成界面工作。

按照 MVVM 分工, View 层遵循以下约束:

约束	说明
只依赖 Common 层	<code>#include "../common/game_state.h"</code> 和 <code>"../common/notification.h"</code> , 不包含 <code>viewmodel/</code> 下的任何头文件
不修改游戏数据	持有 <code>const GameState*</code> 只读指针, 绘制时直接读取, 绝不回写
命令单向注入	所有命令通过 <code>std::function setter</code> 从外部注入, View 内部不创建命令对象
不判断规则对错	不写 AI、碰撞、伤害、刷怪、推图等逻辑, 这些全部属于 <code>ViewModel</code>

作为一个 Qt Widgets 之上的薄层, View 把物理输入 (键盘码) 翻译为已绑定的命令回调, 把只读状态 (`GameState`) 翻译为像素, 把时间节奏 (60 FPS 定时器) 翻译为 `tick` 通知。

与常见的简单 MVVM 视图不同, 游戏 View 还需要处理帧同步与流畅绘制的双重任务——既要保证逻辑帧的稳定递进, 又要确保视觉帧的流畅渲染。

4.2 文件组织与编译

View 层位于 `code/view/` 目录下, 由以下文件组成:

文件	作用
<code>MainWindow.h / MainWindow.cpp</code>	主窗口封装, 提供属性注入、命令注入和通知注册接口
<code>GameWidget.h / GameWidget.cpp</code>	游戏画布, 包含输入处理、定时循环和全部绘制逻辑
<code>InputDefs.h</code>	View 层内部的输入聚合类型 <code>MovementIntent</code>

文件	作用
CMakeLists.txt	编译为 alleyfist_view 静态库，链接 alleyfist_common 和 Qt6::Widgets、Qt6::Gui

编译配置精简且明确：

```
add_library(alleyfist_view STATIC
    MainWindow.cpp
    GameWidget.cpp
)

target_link_libraries(alleyfist_view
    PUBLIC
        alleyfist_common
        Qt6::Widgets
        Qt6::Gui
)
```

View 库不链接 alleyfist_viewmodel，确保 View 与 ViewModel 之间不存在编译级耦合。所有跨层通信均通过 Common 层定义的 GameState、EventNotification 和 std::function 回调完成。

4.3 MainWindow 主窗口

MainWindow 继承自 QMainWindow，内部创建并持有 GameWidget 作为中心控件。它是 View 层对外的入口，App 层通过它完成属性绑定、命令注入和通知注册。

MainWindow 的接口设计直接对应 M-V-VM 三层关系的三种绑定方式：

4.3.1 属性绑定 (Properties)

```
void set_game_state(const GameState* state) noexcept;
```

App 层将 ViewModel 的只读 GameState 指针传入 View。MainWindow 不做任何处理，直接转发给 GameWidget::set_game_state()。View 端在每次绘制时通过 game_state() 辅助函数解引用该指针读取最新状态。

4.3.2 命令注入 (Commands)

MainWindow 对外暴露十一个命令 setter，每个对应一种游戏语义动作：

方法	命令签名	用途
set_tick_command	std::function<void(float, uint64_t)>	帧时间驱动
set_move_left_command	std::function<void(bool)>	左移动（按下/松开）
set_move_right_command	std::function<void(bool)>	右移动
set_move_up_command	std::function<void(bool)>	上移动
set_move_down_command	std::function<void(bool)>	下移动
set_primary_action_command	std::function<void()>	轻攻击
set_secondary_action_command	std::function<void()>	重攻击
set_state_toggle_command	std::function<void()>	跳跃
set_reset_command	std::function<void()>	重新开始
set_confirm_command	std::function<void()>	确认/开始
set_pause_command	std::function<void()>	暂停/恢复

所有命令 **setter** 都是直接将 `std::function` 转发给 `GameWidget` 的对应 **setter**，`MainWindow` 本身不做命令封装。这种透传设计使得 `MainWindow` 保持极薄的职责——仅作为 `App` 层到 `GameWidget` 的命令管道。

4.3.3 通知注册 (Notification)

```
EventNotification get_notification();
```

该方法返回一个 `EventNotification (std::function<void(uint32_t)>)` 闭包。闭包捕获 `this`，在被调用时触发 `GameWidget::update()` 引起重绘。`App` 层在组合根中通过 `viewModel.add_notification(mainWindow.get_notification())` 将闭包注入 `ViewModel`。

当 `ViewModel` 完成状态更新后调用 `fire(kGameStateChangedEvent)` 时，这个闭包被执行，`GameWidget` 随即调度重绘。`GameWidget` 在 `paintEvent` 中通过已绑定的只读指针读取 `ViewModel` 的最新状态，完成从”变化通知”到”数据读取”的解耦闭环。

4.3.4 整体绑定流程

在 `App` 层的组合根中，绑定三个步骤按如下顺序执行：

```
mainWindow.set_game_state(viewModel.get_game_state());

mainWindow.set_tick_command(viewModel.get_tick_command());
mainWindow.set_move_left_command(viewModel.get_move_left_command());
mainWindow.set_move_right_command(viewModel.get_move_right_command());
mainWindow.set_move_up_command(viewModel.get_move_up_command());
```

```
mainWindow.set_move_down_command(viewModel.get_move_down_command());
mainWindow.set_primary_action_command(viewModel.
    ↪ get_primary_action_command());
mainWindow.set_secondary_action_command(viewModel.
    ↪ get_secondary_action_command());
mainWindow.set_state_toggle_command(viewModel.get_state_toggle_command
    ↪ ());
mainWindow.set_reset_command(viewModel.get_reset_command());
mainWindow.set_confirm_command(viewModel.get_confirm_command());
mainWindow.set_pause_command(viewModel.get_pause_command());

viewModel.add_notification(mainWindow.get_notification());
```

这个模式直接参考了课程课件 **Plane** 示例项目中 `AirApp::initialize()` 的绑定方式：

```
m_main_wnd.get_board().set_map(m_game_viewmodel.get_map());
m_main_wnd.set_next_step_command(m_game_viewmodel.get_next_step_command
    ↪ ());
m_game_viewmodel.add_notification(m_main_wnd.get_notification());
```

与 **Plane** 示例的一致性表明，本项目的 **View** 层遵循了标准的 MVVM 绑定范式。

4.4 GameWidget 游戏画布

GameWidget 是 **View** 层的核心，继承自 `QWidget`，约 550 行实现。它同时承担四个职责：**接收数据、收集输入、驱动帧循环和渲染画面**。

4.4.1 数据访问模式

GameWidget 持有 `const GameState* m_gameState` 只读指针，由 **App** 层在绑定时通过 `set_game_state()` 注入。此外持有一个默认构造的 `GameState m_emptyState` 作为后备——当指针为空（尚未绑定）时使用空状态，避免空指针解引用。

通过私有辅助方法 `game_state()` 统一访问：

```
const GameState& game_state() const noexcept
{
    return m_gameState != nullptr ? *m_gameState : m_emptyState;
}
```

所有绘制方法（`drawBackground`、`drawActor`、`drawHUD` 等）通过 `game_state()` 获取快照引用，再访问其中的 `map`、`player`、`enemies`、`hud`、`phase` 等字段。这种间接访问方式有两个好处：一是集中处理空指针边界条件；二是为后续引入帧间插值等高级特性预留了扩展点。

4.4.2 命令注入

GameWidget 持有十一个 `std::function` 成员变量，对应十一种游戏语义命令：

```
std::function<void(float, std::uint64_t)> m_tickCommand;
```

```
std::function<void(bool)> m_moveLeftCommand;
std::function<void(bool)> m_moveRightCommand;
std::function<void(bool)> m_moveUpCommand;
std::function<void(bool)> m_moveDownCommand;
std::function<void()> m_primaryActionCommand;
std::function<void()> m_secondaryActionCommand;
std::function<void()> m_stateToggleCommand;
std::function<void()> m_resetCommand;
std::function<void()> m_confirmCommand;
std::function<void()> m_pauseCommand;
```

每个成员都有对应的 **public setter** (例如 `set_tick_command`、`set_move_left_command` 等)。这些 **setter** 由 **App** 层经由 **MainWindow** 逐层注入 **ViewModel** 的闭包。键盘事件和定时器回调直接调用这些 `std::function`，不经过任何 **Qt** 信号中间层。

这样的设计与参考项目 **Plane** 完全一致：**View** 持有 **ViewModel** 给出的命令闭包，在发生输入事件时直接调用它们，形成从 **View** 到 **ViewModel** 的直接控制通道。

4.4.3 键盘输入处理

GameWidget 重写 `keyPressEvent` 和 `keyReleaseEvent`，将 **Qt** 物理键码映射为已注入的游戏语义命令。按键映射集中在 `isHandledKey` 静态方法和两个事件处理函数中，换手柄或触屏只需修改这里的映射逻辑。

移动键处理 采用”状态聚合 + 命令回调”的模式。**GameWidget** 内部维护 `view::MovementIntent m_movement` 结构体，其中包含四个方向布尔值 `left`、`right`、`up`、`down`。当用户按下移动键时：

1. 检查对应方向标志是否已处于按下状态，避免重复触发
2. 设置方向标志为 `true`
3. 调用已注入的方向命令回调 (如 `m_moveLeftCommand(true)`)

释放移动键时则反向操作——清除方向标志并调用回调 (如 `m_moveLeftCommand(false)`)。这种设计将”哪些键被按住”的状态维护在 **View** 层内部，**ViewModel** 只需要知道每个方向是激活还是停用，无需关心具体哪个物理按键产生了这种变化。

单次触发动作 (攻击、跳跃、暂停、确认、重置) 使用 `m_triggeredThisPress(QSet<int>)` 防止同一按键在按住期间产生多次触发。按下时如果该按键不在集合中，则执行命令回调并将按键加入集合；释放时从集合中移除。这保证了攻击、跳跃等操作的输入语义稳定，避免键盘自动重复导致的意外连击。

按键映射关系如下：

Qt 物理键	触发的命令
--------	-------

← / A	<code>m_moveLeftCommand(true/false)</code>
-------	--

Qt 物理键	触发的命令
→ / D	<code>m_moveRightCommand(true/false)</code>
↑ / W	<code>m_moveUpCommand(true/false)</code>
↓ / S	<code>m_moveDownCommand(true/false)</code>
J / Z	<code>m_primaryActionCommand()</code>
K / X	<code>m_secondaryActionCommand()</code>
Space	<code>m_stateToggleCommand()</code>
R	<code>m_resetCommand()</code>
Enter	<code>m_confirmCommand()</code>
P / Esc	<code>m_pauseCommand()</code>

所有命令回调直接从按键事件中调用，不再经过 Qt 的 signal/slot 机制。这意味着：

- View → ViewModel 的控制通道纯粹由 C++ `std::function` 构成
- Qt 仅作为操作系统事件源 (`QKeyEvent`) 和渲染后端 (`QPainter`)，不参与 MVVM 绑定
- 将来替换为其他 GUI 框架（如 SDL、GLFW）时，只需替换 `QKeyEvent` 到命令的映射逻辑，无需修改 ViewModel 或 Common 层

4.4.4 Tick 与游戏循环

`GameWidget` 内部使用 `QTimer` 以约 60 FPS（16 ms 间隔）的频率驱动游戏逻辑帧。每次定时器触发时：

1. 通过 `QElapsedTimer::restart()` 获取自上一帧以来的真实时间间隔
2. 将 delta time 限制在 0.1 秒以内，防止断点调试或系统卡顿后一次性推进过多逻辑
3. 递增帧序号
4. 累积 GO 闪烁计时（用于 `showGoIndicator` 时的提示闪烁效果）
5. 调用已注入的 `m_tickCommand(clampedDt, m_frameIndex)`

与旧版不同，Tick 不再通过 Qt signal 发出 `tickRequested`，而是直接在定时器 lambda 中调用 `std::function` 回调。`QTimer` 在此仅作为操作系统定时事件的来源——Qt 的信号机制仍然用于将定时器事件从 Qt 事件循环传递到 `GameWidget` 的回调 lambda，但这是 Qt 内部的事件传递机制，与 MVVM 跨层绑定无关。

这种设计使得 View 层的定时器成为一个“纯 View 内部”的时钟源——定时器启动、停止、频率控制全部在 `GameWidget` 内部完成，对外只暴露 `setRunning(bool)` 用于外部启停控制。

4.4.5 坐标转换

ViewModel 输出的是游戏世界坐标：

- x: 关卡横向推进位置 (0 ~ 3000)
- laneY: 街道纵深位置 (300 ~ 500)
- z: 离地高度, 用于跳跃和空中攻击 (0 ~ 90)

View 层通过以下变换将世界坐标映射到屏幕像素:

```
screenX = (worldX - cameraX) * scaleX;
screenY = (laneY - z) * scaleY;
```

其中 cameraX 来自 `game_state().map.cameraX`, 由 **ViewModel** 根据玩家位置和锁屏状态计算。scaleX 和 scaleY 根据标准视口 (960×540) 与当前窗口实际尺寸的比例动态计算, 在 `resizeEvent` 和 `paintEvent` 中更新, 使得游戏画面可以适应不同的窗口大小。

4.4.6 绘制管线

`paintEvent` 是 **View** 层的主绘制入口。整体流程以游戏阶段 `GamePhase` 为分支条件:

```
填充黑色背景
if phase == Title:
    drawOverlay          → 标题画面
else:
    drawBackground       → 夜空 + 视差建筑
    drawStreet           → 人行道 + 车道 + 标线
    按 laneY+z 排序所有可见角色
    drawActor 逐个绘制   → 玩家、敌人
    drawHUD              → 血条、精力条、连击、进度条
    if showGoIndicator:
        闪烁 "GO □" 提示
    if phase == Paused | GameOver | Win:
        drawOverlay      → 半透明遮罩 + 对应文字
```

这种分阶段绘制的方式使得 **View** 可以自然地支持所有游戏界面, 无需额外标志位。

4.4.7 背景与场景绘制

天空: 使用 `QLinearGradient` 绘制从深蓝到地平线亮色的垂直渐变, 模拟夜幕下的城市街道氛围。

远景建筑: 通过硬编码的建筑物位置和尺寸数据, 绘制一系列深灰色矩形作为城市剪影。建筑使用视差滚动——位置随镜头移动的速度仅为主场景的 0.3 倍 (`parallaxFactor = 0.3f`), 增强了横向推进时的空间纵深感。每栋建筑还通过位置哈希绘制明暗交替的窗户网格, 避免静态重复感。

街道: 从 `map.streetTopY` 到 `map.streetBottomY` 分三层: - 上人行道 (15% 高度, 浅灰色) - 车道 (70% 高度, 深灰色), 中央黄色虚线随镜头位置动态偏移 - 下人行道 (15% 高度, 浅灰色) - 路缘石和人行道以下的地面填充

所有街道元素均使用 `QPainter::fillRect` 绘制, 不依赖外部图片资源, 保证在没有图像素材时依然可以完整展示游戏场景。

4.4.8 角色绘制

`drawActor` 方法为单个角色绘制完整形象。它调用 `worldToScreenX` 和 `worldToScreenY` 将世界坐标转换为屏幕坐标后，根据 `Facing`（左右朝向）对画布做水平翻转，再按 `ActorState`（站立、行走、攻击、跳跃、受伤、死亡等）绘制不同姿态。

角色图形完全由 `QPainter` 几何图形构成，包括：

- **腿**：根据状态变化——站立时呈静态、行走时交替前后摆动、跳跃时收起。使用裤子颜色和鞋子颜色区分
- **身体**：矩形躯干，颜色由 `Team`（玩家蓝、小怪红、Boss 深红）和 `ActorKind` 决定。受击时变为红色闪烁
- **手臂**：攻击时向前伸出并带拳头方块，重攻击伸出距离更长。空中攻击时以踢腿姿态展示
- **头部**：圆形头部 + 白色眼白 + 黑色瞳孔 + 队色头巾/头发
- **阴影**：角色底部的地面椭圆阴影
- **特效**：死亡状态红色 X 线、受击状态黄色星形粒子

敌人头顶还会绘制小型血条（绿色 → 黄色 → 红色渐变），帮助玩家判断战斗反馈。无敌状态的角色按 0.1 秒周期交替显示/隐藏。

4.4.9 HUD 绘制

`drawHUD` 将所有界面元素集中在 `View` 层内部：

- **血条 (HP)**：左上角，绿色 → 黄色 → 红色渐变填充，标注 HP 标签和当前值/最大值
- **精力条 (EN)**：HP 条下方，蓝色填充，标注 EN 标签和当前值/最大值
- **疲劳警告**：精力耗尽时显示红色 EXHAUSTED 文字
- **连击计数**：屏幕中上方，白色 1 HIT → 黄色 2 HIT → 红色 3 HIT
- **Boss 血条**：屏幕顶部居中偏右，Boss 出现时显示，标注 BOSS 和血量数值
- **屏幕消息**：中央位置显示 `ViewModel` 传入的提示文字（如 “Paused”）
- **关卡进度条**：底部居中，金色填充进度，根据 `progressRatio` 或玩家位置/世界宽度计算

所有 HUD 元素的颜色、字体和位置均通过 `m_scaleX/m_scaleY` 缩放，保证在不同窗口尺寸下比例一致。

4.4.10 GO 提示与覆盖层

GO 提示：当 `ViewModel` 设置 `map.showGoIndicator = true`（清屏解锁后），`View` 以 0.5 秒亮/0.5 秒灭的节奏在屏幕右上方绘制闪烁的 GO □ 文字。闪烁通过多层描边（3 层不同透明度和宽度的黄色描边 + 白色主体）实现发光效果。

覆盖层：`drawOverlay` 根据 `GamePhase` 绘制四种不同的全屏界面：

阶段	覆盖层内容
Title	深色渐变背景 + 渐变标题 ALLEY FIST + 副标题 + 操作说明列表 + 闪烁 Press ENTER to Start
Paused	半透明黑色遮罩 + PAUSED 大字 + Press P or ESC to Resume
GameOver	半透明遮罩 + 红色 GAME OVER + 存活时间和击败敌人统计 + 闪烁 Press R to Restart
Win	半透明遮罩 + 金色渐变 YOU WIN! + 通关时间和击败敌人统计 + 闪烁 Press R to Restart

标题画面的操作说明直接编码在 View 中，与当前的键盘映射一致，既能作为启动界面引导，也便于测试时快速验证输入映射是否正确。

4.5 InputDefs.h 输入聚合

InputDefs.h 将输入层的类型定义在 View 内部 (namespace alleyfist::view):

```
namespace alleyfist::view {

struct MovementIntent {
    bool left  = false;
    bool right = false;
    bool up    = false;
    bool down  = false;

    bool is_moving() const noexcept
    {
        return left || right || up || down;
    }

    void clear() noexcept
    {
        left = right = up = down = false;
    }
};

} // namespace alleyfist::view
```

MovementIntent 将四个独立方向标志聚合为一个语义结构。这些方向标志在 keyPressEvent / keyReleaseEvent 中被设置和清除。is_moving() 用于快速判断是否有任何方向键被按下，而具体的移动方向推导可由 ViewModel 在收到各方向的 bool 命令后自行完成。

将 MovementIntent 放在 view 子命名空间而非 Common 层，遵循了“输入设备相关类型不应出现在公共契约中”的架构原则。换手柄、触屏等输入方式时，只需修改 GameWidget 的按键映射

和 MovementIntent 的填充逻辑。

4.6 与旧版设计的对比

本项目 View 层的架构经历了两次重要的设计迭代：

4.6.1 第一版：Qt 信号驱动的 MVVM

最初版本采用 Qt signal/slot 作为 MVVM 绑定机制：

```
signals:
    void commandGenerated(const GameCommand& command);
    void tickRequested(float deltaSeconds, std::uint64_t frameIndex);

connect(m_widget, &GameWidget::commandGenerated, this, [this](const
    ↪ GameCommand& command) {
    m_commandSink.handle_command(command);
});
```

这种设计的耦合问题： - GameCommand 需要在 Common 层定义结构体包装输入数据和 Tick 数据 - ViewModel 需要实现 IGameCommandSink / IGameSnapshotSource 接口 - 绑定器 GameWidgetBinding 作为中间人同时持有 View 和 ViewModel 的引用 - Qt 信号机制渗透进 MVVM 绑定路径

4.6.2 第二版（当前版本）：std::function 直接注入

参考课程 Plane 示例后，改为 std::function 直接回调：

对比维度	第一版	第二版
命令传播	emit signal → connect → lambda → handle_command	std::function() 直接调用
命令粒度	一个 GameCommand 结构体 + CommandType 分发	十个独立语义命令，各司其职
接口契约	IGameCommandSink / IGameSnapshotSource 抽象类	std::function setter + const GameState*
通知方式	cookie 回调列表	EventTrigger 的 add_notification
绑定器	独立的 GameWidgetBinding 类	App 层在主函数中直接注入
移动输入	四个独立 Pressed/Released	四个方向 bool 命令 + View 内 MovementIntent 聚合

第二版的关键改进：1. Qt 仅作为事件源(QKeyEvent)和渲染后端(QPainter),不参与 MVVM 绑定 2. 命令签名自文档化——set_move_left_command(std::function<void(bool)>)

明确表达了”向左移动的开/关” 3. View 层不依赖任何 ViewModel 头文件, 只依赖 common/game_state.h 和 common/notification.h 4. 将来替换 GUI 框架只需改 keyPressedEvent/keyReleaseEvent 和 paintEvent, 无需动其他层

4.7 小结

View 层在本项目中的核心贡献是把 Common 契约转化为可玩的 Qt 游戏界面, 同时严格保持自身职责边界:

- **输入:** 键盘物理事件被翻译为已注入的命令回调, 每个方向独立可追踪
- **时间:** 60 FPS 定时器驱动逻辑帧, tick 命令通过 `std::function` 发出
- **状态:** ViewModel 的只读 GameState 指针被解释为可视化的场景、角色和 HUD
- **通知:** ViewModel 的 `fire(kGameStateChangedEvent)` 触发 View 重绘, View 在重绘时从指针拉取最新数据

由于 View 不直接操作规则对象, 也不包含任何 ViewModel 头文件, 后续 ViewModel 继续扩展玩法时, View 只需要响应 GameState 中新增的字段或状态枚举即可。这种分层方式使界面迭代和逻辑迭代可以独立进行, 降低了多人协作时的合并冲突概率。

4.8 技术难点的克服

开发 View 层过程中遇到的主要技术难点及解决方案:

难点一: Common 接口频繁变更导致 View 编译失败。项目初期 Common 层经历了多次重构——ActorViewData 改名为 ActorSnapshot、GameSnapshot 的字段重组 (progressRatio 从子结构移至顶层)、GameCommand 结构体被移除改为 `std::function` 注入。每次 Common 变更都会导致 View 层多处引用失效。解决方式是每次收到 Common 变更通知后, 先用 IDE 全局搜索旧类型名, 逐个定位并更新引用; 同时 View 层内部通过 `game_state()` 辅助函数统一访问入口, 减少了直接引用分散字段的代码量, 降低了后续适配成本。

难点二: Qt 信号到 `std::function` 的模式迁移。第一版 View 使用 Qt signal/slot 做 MVVM 绑定——GameWidget 通过 `emit commandGenerated()` 和 `emit tickRequested()` 发出信号, GameWidgetBinding 用 `connect` 桥接到 ViewModel。老师指出这与 Plane 示例的 `std::function` 回调模式不一致。改造的挑战在于: 键盘事件遍布 `keyPressEvent` 和 `keyReleaseEvent` 两处, 定时器回调在构造函数 `lambda` 中, 每处都需要从 `emit signal(...)` 改为 `if (m_xxx) m_xxx(...)`, 且要保证每个 `std::function` 都有对应的 `setter` 和空指针安全检查。解决方式是先完成 MainWindow 层的接口设计 (十一个独立命令 `setter`), 再逐个替换 GameWidget 中的 `emit` 调用, 最后删除 GameWidgetBinding 和所有 `signals:` 声明。

难点三: 键位去重与状态聚合。游戏需要区分”持续按住方向键移动”和”单次按下攻击/跳跃”。Qt 键盘事件自带的 `isAutoRepeat()` 无法完全解决问题: 玩家快速连按同一攻击键时, 如果键位没有完全释放就再次按下, 会导致第二次攻击被忽略。解决方案是用 `QSet<int>`

`m_triggeredThisPress` 记录本轮已触发的按键，只有在 `keyReleaseEvent` 清除记录后，同一按键才能再次触发。对于移动键，通过 `MovementIntent` 结构体聚合四个方向，避免对同一方向重复发送 `Pressed` 命令。

难点四：无图片素材下的游戏画面呈现。项目没有使用任何外部 PNG 图片资源，所有视觉元素——玩家、敌人、Boss、建筑、街道、HUD 血条、GO 提示——全部用 `QPainter` 几何图形绘制。挑战在于用有限的矩形、圆形和线条组合出可辨识的角色形象和场景氛围。解决方案包括：用 `QLinearGradient` 模拟夜空和文字渐变、用视差系数（0.3x）区分远景建筑和主场景的深度、用正弦函数驱动行走时的腿部摆动、用不同颜色方案区分玩家/小怪/Boss、为受伤和死亡状态设计专门的视觉反馈（红色闪烁、黄色星效、红色 X 线）。

5 ViewModel 层分报告

负责人：邱安琪 学号：3240105004

5.1 层次定位

本工程遵循面向游戏类项目的 MVVM 实践规范。由于游戏场景中的核心数据（如实体状态、位置等）直接对应视图层的可绘制对象，无需进行二次业务模型转换，因此本架构合并了传统意义上的 Model 层，将其统一内聚于 ViewModel 层中。ViewModel 层负责把游戏规则封装为独立的模拟器（simulation）并将可绘制的状态快照提供给 View。View 只负责呈现和收集输入，所有玩法逻辑（移动、碰撞、伤害、敌人 AI、Boss 产生与判定等）都在 Simulation 中实现，且不依赖于 Qt 或绘图 API。

在实现上该层由两部分组成：

- GameViewModel：对外门面，负责命令绑定、状态同步和事件通知；
- GameSimulation：内部仿真器，维护 EntityState 列表并推进移动、攻击、敌人 AI 与 Boss 逻辑。

这种设计的优势在于：一方面避免了 Model 到 ViewModel 的冗余数据转换，另一方面使得核心逻辑完全不依赖 Qt 或绘图 API，因而可以在无界面环境下进行逻辑验证与后续扩展。

5.2 文件组织与编译

ViewModel 层位于 code/viewmodel/ 目录下，主要由以下文件组成：

文件	作用
GameViewModel.h / GameViewModel.cpp	对外暴露命令接口、维护本地状态、负责与 View 同步 GameState
GameSimulation.h / GameSimulation.cpp	实现规则引擎，处理实体移动、攻击、AI、Boss 生成与胜负判定
SimulationTypes.h	定义 EntityState、EntityList、实体种类和动作相关类型
CMakeLists.txt	编译为 alleyfist_viewmodel 静态库

编译配置如下：

```
add_library(alleyfist_viewmodel STATIC
    GameViewModel.cpp
    GameSimulation.cpp
)

target_link_libraries(alleyfist_viewmodel
```

```

    PUBLIC
        alleyfist_common
    )

```

与 View 层相比, ViewModel 层不直接链接 Qt 库, 而是通过 Common 层提供的公共契约与 View 进行通信。这样可以保证逻辑层与界面层的解耦, 后续无论替换为 SDL、OpenGL 还是其他框架, 都不会影响游戏规则本身。

5.3 GameViewModel

GameViewModel 是整个 ViewModel 层对外暴露的核心入口。它不直接参与具体规则计算, 而是负责把 View 的输入转成内部命令, 并将规则结果整理为 View 需要的状态快照。

5.3.1 核心职责

GameViewModel 的职责可以概括为三点:

1. 接收 View 发送的命令, 例如移动、攻击、跳跃、暂停、确认和重置;
2. 维护本地动作状态, 例如按键状态、跳跃计时、攻击计时和玩家能量;
3. 将 GameSimulation 的结果同步到 GameState, 并通过事件通知 View 刷新界面。

5.3.2 命令注入与回调

当前工程采用的是轻量级的“命令回调”风格, 而不是传统意义上的完整命令类层次结构。GameViewModel 通过 `std::function` 暴露命令接口, 例如:

```

std::function<void(float, std::uint64_t)> get_tick_command();
std::function<void(bool)> get_move_left_command();
std::function<void()> get_primary_action_command();
std::function<void()> get_pause_command();

```

View 层只需要注册这些回调即可, 在收到输入后, ViewModel 再由内部方法处理具体逻辑。比如 `make_move_command()` 用来维护方向键状态, `begin_attack()` 则负责触发攻击并消耗能量。这种设计使 View 不必知道“攻击是如何发生的”, 只需要发出“攻击”请求即可。

5.3.3 Tick 驱动与状态同步

在 `code/viewmodel/GameViewModel.cpp` 的 `GameViewModel::tick()` 中, 具体实现是这样的:

```

m_state.elapsedSeconds += dt;
update_action_timers(dt);

const float dx = (m_moveRight ? kMoveSpeed : 0.0f) - (m_moveLeft ?
    ↪ kMoveSpeed : 0.0f);
const float dy = (m_moveDown ? kMoveSpeed : 0.0f) - (m_moveUp ?
    ↪ kMoveSpeed : 0.0f);
if (dx != 0.0f || dy != 0.0f) {

```

```
m_sim->player_move(dx * dt, dy * dt);  
}  
  
m_sim->step(dt);  
sync_state_from_simulation();  
notify_state_changed();
```

这段代码直接体现了 **ViewModel** 的职责链路：先更新局部状态，再推进仿真，再生成快照，最后通过 `fire(kGameStateChangedEvent)` 通知 **View**。与此同时，`sync_state_from_simulation()` 还会把玩家位置限制在 `0.0f ~ kWorldWidth` 和 `kStreetTop ~ kStreetBottom` 之间，确保内部数据不会越界。

5.4 GameSimulation 仿真核心

GameSimulation 是整个 **ViewModel** 层的规则引擎，负责实体状态、战斗逻辑和关卡推进。

5.4.1 实体与世界状态

仿真层使用 **EntityState** 维护每个实体。玩家为列表第一个实体，后续实体为敌人或 **Boss**。每个实体包含位置、血量、最大血量、是否存活等信息。这样的设计使得逻辑判断比较直观：所有规则都围绕一组实体进行，而不是围绕界面对象进行。

```
struct EntityState {  
    alleyfist::WorldPosition pos;  
    EntityKind kind = EntityKind::Grunt;  
    int hp = 0;  
    int maxHp = 0;  
    int id = 0;  
    bool alive = true;  
};  
  
using EntityList = std::vector<EntityState>;
```

5.4.2 移动与战斗

在 `code/viewmodel/GameSimulation.cpp` 中，规则逻辑是直接写在几个关键函数里的：

- `player_move()` 会对玩家位置做边界限制：`p.pos.x = std::clamp(p.pos.x + dx, 0.0f, kWorldWidth);`, `p.pos.laneY = std::clamp(p.pos.laneY + dy, kStreetTop, kStreetBottom);`
- `simulate_ai()` 会根据敌人类型决定移动速度：`float speed = e.kind == EntityKind::Boss ? 55.0f : 30.0f;`，然后让敌人朝玩家方向靠近；
- `player_attack()` 会对附近敌人进行攻击判定，并执行 `e.hp -= 10;`，若敌人血量归零则设置 `e.alive = false;`。

普通小怪和 **Boss** 的战斗表现并不相同：小怪靠近玩家后对其造成较小伤害，而 **Boss** 的近距离

伤害更强，且命中范围也更大。这些细节都直接体现在 `simulate_ai()` 和 `player_attack()` 的判断逻辑里。

5.4.3 Boss 与胜负判定

为了增强关卡感，仿真器还内置了 Boss 触发逻辑：

```
constexpr float kBossTriggerX = 2350.0f;
constexpr float kBossSpawnOffsetX = 260.0f;
constexpr int kBossHealth = 140;
```

`spawn_boss_if_needed()` 会在玩家位置满足 `player.pos.x >= kBossTriggerX` 时生成 Boss，并把 `boss.pos.x` 设置为 `std::min(kWorldWidth - 80.0f, player.pos.x + kBossSpawnOffsetX)`；Boss 的初始血量为 `kBossHealth`。随后 `update_boss_state()` 会检查 `std::any_of(...)` 是否仍存在存活的 Boss，并把 `m_bossDefeated` 更新为 `true`。最后在 `code/viewmodel/GameViewModel.cpp` 的 `sync_state_from_simulation()` 中，若 `m_sim->boss_defeated()` 为真，就把阶段切换为 `GamePhase::Win`。

5.5 状态快照与阶段切换

`sync_state_from_simulation()` 是 `ViewModel` 向外输出状态的关键函数。它会把内部实体状态整理为 `GameState`，包括以下内容：

- 玩家位置、血量、能量、朝向和动作状态；
- 敌人列表与 Boss 血条信息；
- 地图相机位置、推进比例和 HUD 状态；
- 当前阶段（Playing、Paused、GameOver、Win）。

这一层的设计使 `View` 可以保持相对简单：它只需要读取一份快照，不需要自己去推断角色位置、敌人数量或阶段状态。对应地，`ViewModel` 也不需要知道屏幕上如何绘制，只需要保证快照字段语义明确即可。

在具体实现中，`sync_state_from_simulation()` 还会根据玩家位置计算 `progressRatio`，即 `player.x / 3000.0f` 的归一化值；同时根据玩家位置和视口尺寸计算 `cameraX`，公式为 `player.x - 960 * 0.42`，从而实现镜头的平滑跟随。这样，`View` 层只需直接读取 `GameState.map.cameraX` 与 `GameState.progressRatio`，就能得到场景推进和 HUD 进度的显示数据。

其中阶段切换逻辑为：

- 玩家死亡进入 `GameOver`；
- Boss 死亡进入 `Win`；
- 暂停时切换为 `Paused`；
- 确认键可在标题、失败或胜利状态下重开游戏。

5.6 小结

本工程的 ViewModel 层实现了一个简洁但完整的”输入—规则—状态”链路：

- GameViewModel 负责输入接收、状态同步与事件通知；
- GameSimulation 负责纯规则逻辑，包括移动、攻击、敌人 AI、Boss 生成与胜负判定。

这种结构使 View 只需关注显示与交互，而玩法逻辑则被集中封装在 ViewModel 层内，便于后续继续扩展更复杂的关卡与战斗系统。

5.7 技术难点的克服

开发 ViewModel 层过程中，主要遇到以下技术难点：

难点一：输入与逻辑的解耦。最初版本在 Common 层定义了 InputAction 枚举和 GameCommand 结构体，View 发出 GameCommand，ViewModel 通过 process_input_command() 做 switch-case 分发。这种设计的耦合问题在于：Common 层被迫包含输入设备相关的概念 (MoveLeft、LightAttack 等)，而 ViewModel 也需要理解按下/释放的状态语义。在袁老师指导下，我们将命令改为 std::function 回调注入——get_move_left_command() 返回 std::function<void(bool)>, get_primary_action_command() 返回 std::function<void()>。ViewModel 不再需要知道”按键”这个概念，只需要响应”左移动开启/关闭”这样的语义信号。

难点二：Tick 与状态同步的一致性。每帧的逻辑推进涉及多个步骤：更新动作计时器、处理移动输入、调用 GameSimulation::step()、同步内部实体到 GameState 快照、触发通知。如果顺序不统一，容易出现画面滞后或逻辑状态不同步。解决方案是将 tick() 设计为统一入口，严格按”计时器更新 → 输入处理 → 模拟推进 → 状态同步 → 通知触发”的顺序执行，保证每帧的状态一致性。

难点三：动作系统与能量系统的协调。跳跃、攻击、能量恢复和疲劳状态需要同时管理——玩家不能在被打硬直时跳跃，不能在空中连续跳跃，能量耗尽后需要恢复期。解决方案是通过 m_jumpElapsed、m_attackTimer、m_playerEnergy、m_exhaustedWarningTimer 以及 try_spend_energy() 组成一组局部状态管理，is_gameplay_active() 和 is_player_action_locked() 作为守卫条件，在命令入口处拦截不合法的操作。

难点四：Boss 与胜负条件的判定。规则系统一开始较简单，但当 Boss 生成、敌人死亡和玩家胜负条件同时存在时，逻辑容易变得混乱。最终通过 spawn_boss_if_needed()、update_boss_state() 与 boss_defeated() 的分层处理，成功把 Boss 触发、存活检测和胜负切换拆解成明确的步骤。

6 个人反思与收获

6.1 苏易文 - App/Common 层

6.1.1 负责内容

App/Common 层，包括公共类型、游戏状态、通知工具、应用组合根、生命周期管理和三类绑定。

6.1.2 个人反思与收获

本次项目中我主要负责 App/Common 层。开发过程中我最深的体会是：公共层不能什么都放，Common 层越克制，后续协作越轻松。

一开始我们曾经倾向于在 Common 层定义比较完整的命令和快照接口，例如把输入动作、Tick、状态快照和绑定接口都统一抽象出来。这个方案看起来“完整”，但实际联调时会发现它容易让 Common 层承担过多职责：View 的输入细节、ViewModel 的命令粒度、状态同步方式都会被提前固定下来。后来参考课程示例后，我把公共契约收敛为三类更稳定的内容：types.h 中的基础值类型、game_state.h 中的只读显示状态、notification.h 中的轻量事件通知。输入命令则由 ViewModel 通过 `std::function` 暴露，App 层负责注入给 View。这样 Common 层只保留真正需要跨层共享的内容，边界比之前清楚很多。

在 GameState 的设计上，我也更理解了“给 View 的状态”和“ViewModel 内部状态”之间的区别。View 需要知道玩家位置、血量、动作状态、镜头位置、敌人列表和 HUD 数据，但不应该知道敌人追踪速度、Boss 触发阈值、攻击命中范围、跳跃计时等内部规则。把这些模拟细节留在 ViewModel 内部，可以避免 View 因规则调整频繁改动；而把绘制必需的数据整理进 GameState，又能让 View 不需要到处拼接内部对象。这个过程让我认识到，公共数据结构不是越详细越好，而是要刚好表达跨层协作所需的信息。

App 层的 GameApplication 让我真正体会到组合根的价值。程序入口 main.cpp 只创建 GameApplication 并调用 `run()`，具体的 QApplication、MainWindow 和 GameViewModel 都被藏在 PImpl 里。这样公开头文件不会泄漏 Qt 和具体模块类型，也避免了其他层绕过 App 直接拿到内部对象。所有 properties / commands / notification 绑定都集中在 GameApplication::Impl 构造函数中完成，调试绑定问题时只需要看一个地方，项目结构也更容易向队友解释。

这次协作中另一个收获是，接口稳定性对多人开发非常重要。View 层和 ViewModel 层同学都依赖 Common/App 提供的边界，如果我频繁修改命名或数据结构，就会给他们带来额外适配成本。因此后期我更注意在改接口前先确认它是否真的属于公共契约，是否可以用更小的类型表达，是否会迫使其他层知道不该知道的细节。这个思考过程比单纯写代码更有工程实践意义。

总体来说，App/Common 层让我对 MVVM 有了更进一步的理解。一个好的分层不是把代码机械拆开，而是让每一层只看到自己应该看到的东西。Common 提供稳定语言，App 完成对象组装，View 和 ViewModel 才能相对独立地演进。后续继续完善项目时，我会更加重视公共接口的稳定性和跨层边界。

6.2 陈棋隆 - View 层

6.2.1 负责内容

View 层，包括 MainWindow、GameWidget、键盘输入、定时器、绘制、HUD 和覆盖层。

6.2.2 个人反思与收获

对 MVVM 模式的理解从”知道”变为”用过”。课程上讲的 MVVM 概念——数据绑定、命令模式、属性通知——在本次项目中全部落地为具体代码。特别是从第一版 Qt 信号方案切换到第二版 `std::function` 注入方案的过程，让我真正理解了”用什么机制做绑定”比”绑了什么”更影响架构质量。Qt 信号虽然方便，但它把 MVVM 绑定路径和特定 GUI 框架耦合在一起；`std::function` 回调则是纯 C++ 机制，View 和 ViewModel 都只依赖标准库。

View 层的约束比想象中更重要。开发初期，容易在 View 的绘制代码中顺手加入一些判断逻辑（如”如果玩家血量低于 20% 就闪烁红色边框”）。后来意识到这种判断属于 ViewModel 的职责——血量阈值的含义应该由规则层定义，通过 GameState 的字段传达给 View。此后严格遵守”View 只读取状态，不判断状态含义”的原则，所有颜色切换、闪烁频率、显示/隐藏条件都由快照中的字段决定。

Draw 比想象的更有挑战。在没有图片素材的条件下用 QPainter 手绘全部游戏画面，需要综合考虑角色辨识度、颜色方案一致性、不同状态下的姿态差异、缩放适配等。这是一次对图形 API 基本功的集中锻炼。

6.3 邱安琪 - ViewModel 层

6.3.1 负责内容

ViewModel 层，包括 GameViewModel、GameSimulation、移动、攻击、敌人 AI、Boss 生成和胜负判定。

6.3.2 个人反思与收获

在本次项目中，我负责 ViewModel 层的核心规则引擎和状态同步逻辑。MVVM 中 ViewModel 与 View 解耦的思想让我能够专注于游戏推进逻辑，也使工程更容易扩展和维护。在与袁老师交流后，我纠正了早期设计中的一些误区，例如没有提前考虑扩展性，导致 ViewModel 和 View 几乎一对一绑定。工程一旦变复杂，这种做法就只能依靠不断增加绑定接口维持，后续修改会比较困难。

与此同时，Tick 驱动的推进方式让我进一步理解了游戏逻辑与渲染更新的关系：每一帧由 ViewModel 收集输入并调用 Simulation 进行一步推进，再把结果整理为统一的 GameState 快照，从而保证界面显示与内部规则始终保持一致。

由于之前没有相关经验，我先学习了状态机、碰撞判定和攻击盒等游戏逻辑，再一步步应用到项目中。看到代码最终变成可以实际操作的游戏，让我很有成就感。

7 总结与展望

7.1 架构收获

本实验最重要的收获是通过 MVVM 把“输入、规则、显示”拆开。最初找袁老师验收时，我们的代码耦合度非常高：键盘事件里直接改坐标，绘制函数里顺手判断攻击，敌人 AI 和窗口状态混在一起。经袁老师指出，这样的代码很难并行开发，后续迭代和维护都会变得非常困难。

在袁老师的指导下，我们进行了四次改进，最终通过了袁老师的验收。当前实现将跨层共享内容收敛为 GameState、基础值类型和通知工具，并把命令设计为从 ViewModel 暴露出来的 `std::function` 回调。ViewModel 负责规则，View 只根据只读状态完成绘制。如此一来，每个人可以围绕自己的层次推进，不必频繁修改其他人的内部实现。

另一个收获是组合根的重要性。GameApplication 集中创建 Qt 应用、主窗口和 ViewModel，并在一个地方完成 properties / commands / notification 绑定。这样 main 函数、View 和 ViewModel 都不需要知道完整对象组装细节，依赖方向更加清楚。

7.2 项目整体进展的经验体会

回头看这段时间的开发，项目是从最开始只能打开窗口、显示简单画面，一点点做到现在可以移动、攻击、打 Boss，并且有暂停、失败、胜利和重新开始等完整流程的。虽然目前离一款内容丰富的游戏还有距离，但至少已经做出了一个可以实际游玩的原型。对我们来说，这个过程不只是把几个功能拼在一起，更重要的是把输入、游戏逻辑、状态更新和界面绘制真正连接了起来。

开发过程中比较明显的一个体会是，功能不能一下子铺得太多。刚开始我们先保证窗口、Tick 更新、角色移动和画面绘制能够正常工作，之后才陆续加入攻击、精力、敌人 AI 和 Boss 等内容。这样做虽然看起来慢一些，但每次出现问题时比较容易确定是哪一次修改造成的。如果一开始就同时做复杂战斗、动画和关卡，很可能一个错误会影响很多地方，反而更难调试。

项目的架构也不是一开始就完全确定的。虽然目录上采用了 MVVM 分层，但实际开发后还是出现过 Common 层内容太多、接口命名反复变化、View 和 ViewModel 绑定过于直接等问题。后来大家根据各自模块的实际需求重新整理接口，把 View 需要的数据统一放进 GameState，把具体规则留在 ViewModel，再由 App 层集中完成命令和通知的绑定。经过这些调整以后，各层负责什么、不能做什么都清楚了很多。我们也体会到，重构不一定代表前面的代码白写了，有时只有真正用起来，才能发现原来的设计哪里不合适。

多人合作方面，前期公共接口变化比较频繁，其他成员经常需要跟着修改，这部分确实产生了一些额外工作。后期我们开始更注意提前沟通字段含义和命令名称，并通过代码注释、README 和 Git 提交说明修改内容，联调效率比之前高了不少。如果以后再做类似项目，我们会尽量更早列出公共接口和阶段目标，同时增加相互检查代码的过程，而不是等到最后合并时才集中发现问题。

目前项目还有不少可以继续改进的地方，例如自动化测试比较少，输入、界面和跨层绑定大多还

是依靠人工运行检查；角色动作、攻击反馈、音效和关卡内容也比较基础。不过，通过这次实验，我们已经实际经历了从分工开发到合并、调试和完善的完整过程，也更加理解了一个项目能够稳定运行，除了实现功能之外，还需要清晰的结构、及时的沟通和持续的检查。这些经验对我们之后继续完成项目会有很大帮助。

7.3 智能体的使用

本项目的需求分析、任务分工、架构选择、核心代码实现和最终联调均由三位成员共同完成。开发过程中我们使用了 Claude Code 智能体，但它的定位始终是辅助工具，而不是项目的设计者或完成人。每次使用前，都是由成员先明确要解决的问题和修改范围；智能体给出结果后，再由对应模块的负责人阅读、修改、编译和运行验证，确认没有问题后才会保留到项目中。

智能体主要用在以下几类工作中：

- **辅助处理重复代码。**在接口已经由成员确定后，智能体可以协助补充部分 setter、getter、命令绑定和绘制代码骨架。它减少了重复输入的时间，但角色如何绘制、HUD 如何布局、命令应该怎样划分，仍然由负责对应模块的成员决定。
- **辅助搜索和重构。**Common 层类型或字段发生变化时，我们利用智能体查找旧名称的引用，并给出需要同步修改的位置。成员会逐项检查这些修改，避免误改其他变量或破坏原有逻辑。公共接口最终保留哪些内容，也是三位成员讨论后确定的。
- **辅助排查问题。**遇到编译错误或跨层绑定问题时，智能体可以帮助整理可能原因，例如头文件依赖、函数签名不一致或命令遗漏等。不过，真正的问题需要成员结合代码、编译信息和运行结果定位，不能只根据智能体的回答判断。
- **辅助整理报告。**智能体参与了部分章节初稿、代码说明和文字结构的整理。报告中的项目过程、成员分工、技术难点和个人体会均来自我们的实际开发经历，生成的文字也经过了成员修改，避免出现与代码不符或表达过于模板化的内容。

从实际效果看，智能体对批量修改、代码搜索和初稿整理比较有帮助，节省了一些机械性工作的时间，也能在遇到问题时提供新的排查思路。这样三位成员可以把更多精力放在 MVVM 分层、公共接口、游戏规则、画面效果和模块联调等需要人工判断的工作上。项目最终采用的架构、各层职责和游戏功能，都是成员在多次讨论、修改和验收后确定的，并不是直接使用智能体一次生成的结果。

在实际使用中，我们也发现了智能体存在以下问题：

- **命名不一定准确。**智能体有时会根据局部代码直接取名，而没有充分考虑类型在整个项目中的含义。例如，它曾经倾向于把 Common 层中的 GameState 命名为 GameViewState。这个状态虽然会被 View 读取，但它本质上是 View 和 ViewModel 共同使用的跨层状态；加入 View 后容易让人误以为该类型只属于界面层，反而造成职责上的歧义。
- **不能很好地理解 MVVM 边界。**智能体能够按照要求生成能运行的代码，但不一定真正理解各层为什么要这样划分。生成结果中曾出现把输入细节放入 Common、让 View 直接处理游戏规则，或者让 ViewModel 依赖界面类型等情况，导致比较严重的层间耦合。这些问题通常不会马

上表现为编译错误，而是在后续修改和联调时才暴露出来。

- **容易出现便宜模式。**智能体为了尽快完成当前要求，有时会直接暴露内部对象、增加临时接口或绕过原有分层。这样的代码可能暂时解决问题，但会破坏依赖方向，并把维护成本留到后面。因此，不能只根据“现在能不能运行”判断生成结果是否合适。
- **复杂逻辑容易越写越绕。**当提示中同时包含较多条件时，智能体可能不断增加分支、状态变量和补丁式判断，最后形成层层嵌套、重复判断较多的逻辑。虽然功能表面上能够实现，但代码可读性和可维护性较差，需要成员重新梳理数据流、拆分函数，有时甚至需要重新设计后再实现。
- **对使用者的意见过于顺从。**当我们提出一个方案时，智能体往往会先表示认可，再沿着这个方向继续生成代码，即使方案本身可能存在架构或逻辑问题。它较少主动质疑需求中的错误前提，也不一定比较不同方案的代价。因此，成员不能把智能体的认同当作方案正确的证明，而应结合课程要求、代码结构、运行结果和团队讨论独立判断。

正因为存在这些问题，我们始终把智能体的输出当作参考或初稿，而不是可以直接使用的最终答案。架构边界、类型命名、功能逻辑和最终代码都由三位成员共同检查，并通过编译、运行、试玩和老师验收不断修改。智能体提高了部分工作的效率，但项目质量最终仍然取决于成员自己的理解、判断和协作。

7.6 后续改进方向

当前项目已经具备核心玩法骨架，但仍有一些可以继续完善的方向：

- 将敌人、Boss 触发和战斗数值从硬编码常量抽离成配置表，方便后续调整难度。
- 增加更完整的连招链、击退、击倒、无敌帧和受击硬直。
- 补充遭遇战锁屏、清屏解锁和 GO 提示字段，使关卡推进更接近最终设计。
- 引入图片、精灵动画和音效资源，替代当前的基础 QPainter 几何绘制。
- 补充单元测试，覆盖移动边界、攻击命中、Boss 生成、胜负判定等规则。
- 增加攻击命中的画面反馈——屏幕震动、受击粒子、伤害数字飘字。

7.7 总结

本阶段完成了一个可运行且架构正确的横版动作游戏原型。项目从工程组织、公共状态、命令回调、界面绘制到规则模拟都已经形成闭环。通过 MVVM 分层和 `std::function` 命令注入，View 和 ViewModel 保持了清晰的单向依赖，为后续功能扩展和界面升级奠定了扎实的架构基础。